Metody optymalizacji

Zadanie 0

Implementacja i badanie Algorytmów Ewolucyjnych w porównaniu do innych metod nieewolucyjnych

Kontakt: pawel.myszkowski@pwr.edu.pl

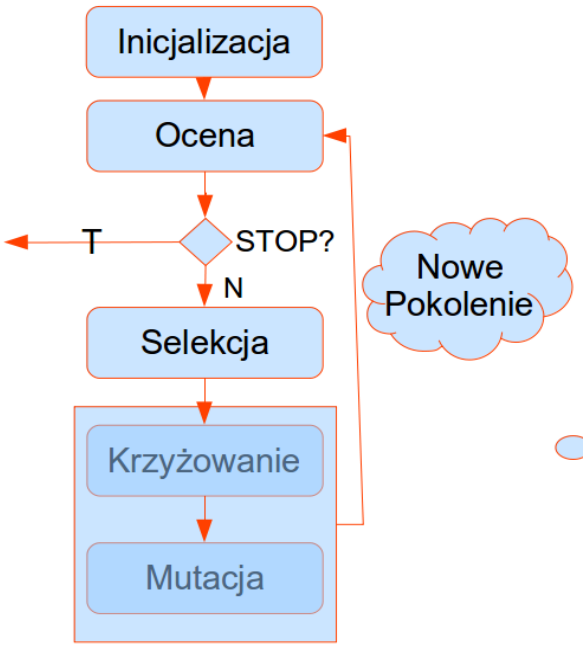
aktualizacja: 05.03.2026

Cel ćwiczenia

Zapoznanie się z metaheurystyką algorytmu ewolucyjnego w praktyczny sposób poprzez samodzielną implementację. Algorytm ewolucyjny jest metaheurystyką (choć nazwa inaczej sugeruje – nie jest to algorytm – to jest metaheurystyka), która naśladuje ewolucję naturalną metodą ciśnienia selekcyjnego i doboru naturalnego. Aby ją zastosować, należy zdefiniować potencjalne rozwiązanie (osobnika), sposoby jego modyfikacji (mutacja), łączenia (krzyżowania) oraz oceny jakości rozwiązania (funkcja oceny).

Mała wyjaśnienie odnośnie terminologii używanych metod.
Algorytm, wedle definicji jest to skończona sekwencja operacji pozwalający w skończonym czasie rozwiązać problem. Czas działania algorytmu zależy od rozmiaru danych wejściowych (tj. złożoność obliczeniowa algorytmu) i gwarantuje znalezienie rozwiązania optymalnego. W przypadku **heurystyki** złożoność obliczeniowa może być różna (zależy od pomysłowości programisty) i nie gwarantuje znalezienia optymalnego rozwiązania. W tym kontekście **metaheurystyką** nazwiemy metodę, która jest heurystyką wyższego poziomu, tj. heurystyką która określa jakie/jak stosować heurystyki niższego poziomu. Przykładem heurystyki niższego poziomu jest inicjalizacja, selekcja, krzyżowanie itp.
Kiedy stosujemy (meta)heurystyki? Przede wszystkim, wtedy kiedy nie mamy algorytmu.

Algorytm ewolucyjny (EA) opiera się na schemacie przedstawionym na schemacie 1 (patrz Rysunek 1) i w pseudokodzie 1. EA wstępnie inicjalizuje (zwykle losowo) populację rozwiązań. Następnie ocenia jakość poszczególnych osobników. W kolejnym kroku sprawdzane są warunki zatrzymania: czy osiągnięto akceptowalne rozwiązanie i/lub limit pokoleń (ew. liczby obliczeń funkcji oceny) został przekroczony. Wybór osobników (sugerowana metoda turnieju lub ruletki) do następnej populacji następuje przed działaniem operatora krzyżowania i mutacji, które budują osobniki nowego pokolenia. Dalej, następuje ocena nowych rozwiązań i cykl EA się zamyka przy sprawdzaniu warunków zatrzymania.



Rysunek 1. Ogólny schemat działania Algorytmu Ewolucyjnego

```
begin
t:=0;
initialise( pop(t0) );
evaluate( pop(t0) );
      while (not stop_condition) do
      begin
        pop(t+1) := selection( pop(t) );
        pop(t+1) := crossover( pop(t+1) );
        pop(t+1) := mutation( pop(t+1) );
        evaluate( pop(t+1) );
        t:=t+1;
      end
return the_best_solution
end
```

Pseudokod 1. Ogólny pseudokod Algorytmu Ewolucyjnego

Na Pseudokod 2 przedstawiono inny sposób budowy przepływu osobników w Algorytmie Ewolucyjnym. Wykorzystano tam operatory krzyżowania tworzące jednego osobnika, jednak bez większych przeszkód ten pseudokod może być rozszerzony dla operatorów zwracających dwa osobniki.

```
begin
t:=0;
initialise( pop(t) );
evaluate( pop(t) );
      while (not stop_condition) do
      begin
        while ( pop(t+1).size()!=pop_size )
            P1 := selection( pop(t) );
```

```
            P2 := selection( pop(t) );
            if ( rand[0,0..1,0] < Px)
              O1:= crossover( P1, P2 );
            else O1:=P1;
            O1 := mutation( O1, Pm );
            evaluate( O1 );
            pop(t+1).add( O1 );
            if ( the_best_solution > O1 )
                    the_best_solution:=O1
        end
      t:=t+1;
      end
return the_best_solution
end
```

Pseudokod 2. Pseudokod jako przepływ osobników dla Algorytmu Ewolucyjnego

W Pseudokod 2 populacja bieżąca *Pop(t)* stanowi podstawę do operacji dla selekcji – wybierane są osobniki *P1* i *P2*, a następnie sprawdzane jest prawdopodobieństwo krzyżowania. Jeśli zachodzi, operacja jest wykonywana a osobnik potomny *O1* jest mutowany. Aby nie „marnować" przebiegu EA, w przypadku braku operacji krzyżowania osobnik *O1* jest kopiowany od rodzica *P1*, a dalej mutowany. W dalszej kolejności podlega ocenie i dodania do populacji następnej – *Pop(t+1)*. Procedura może być łatwo rozbudowana o operację krzyżowania, która zwraca dwa osobniki potomne.

W ramach zadania należy napisać własną metodę Algorytmu Genetycznego (AG). AG służy do wyszukiwania jak najlepszych rozwiązań dla problemów optymalizacji. Niniejsze zadanie jest zadaniem wstępnym, nie ma na celu pokazania całego spektrum możliwych zastosowań AG. Może jednak stanowić intelektualną rozrywkę dla osób, którym spodoba się ta tematyka.

Niniejsze zadanie, jest zadaniem wstępnym. Służy do zapoznania się ze sposobem wykonywania badań w zakresie optymalizacji. Do rozwiązania jest problem PFSP (ang. *Permutation Flowshop Scheduling Problem*), a rozwiązania tego problemu kodują kolejność, ale też można do tego wykorzystywać wektor liczb całkowitych lub rzeczywistych.

Przy wszystkich zadaniach realizowanych na laboratorium rozwiązywany jest permutacyjny problem przepływowy szeregowania zadań **PFSP** (ang. ***Permutation Flowshop Scheduling Problem***). Problem PFSP jest problemem NP-trudnym i ze względu na jego trudność do jego rozwiązywania używane są metaheurystyki.

Problem PFSP definiowany jest przez zbiór M maszyn, J zadań oraz macierzy o wymiarach $M \times J$ zawierającej czasy potrzebne do wykonania j-tego zadania na m-tej maszynie, gdzie $j \in \{1, \dots, J\}$ i $m \in \{1, \dots, M\}$. W problemie PFSP, każde zadanie musi zostać zrealizowane na każdej maszynie, a kolejność maszyn, na których realizowane są zadania jest z góry określona, tj. realizacja zadania na danej maszynie może rozpocząć się dopiero po zakończeniu wykonywania tego samego zadania na poprzedniej maszynie, i jest taka sama dla wszystkich zadań. Każda maszyna może realizować tylko jedno zadanie w tym samym czasie.

W ramach zadania wstępnego szukamy takiej sekwencji wykonywania zadań x na każdej maszynie:

$$x = (x_1, \dots, x_J),$$

która minimalizuje całkowity czas potrzebny na realizację wszystkich zadań na wszystkich maszynach z uwzględnieniem czasu oczekiwania na wolną maszynę. Funkcja celu określona jest zatem następująco:

$$f(x_1, \dots, x_J) = \sum_{j=1}^{J} c(x_j, M),$$

gdzie $c(x_j, m)$ oznacza czas ukończenia x_j-tego zadania na m-tej maszynie. Wartość $c(x_j, m)$ wyznaczamy zgodnie z następującym wzorem:

$$c(x_j, m) = \begin{cases} p(x_j, m) & , j = 1 \wedge m = 1 \\ c(x_j, m-1) + p(x_j, m) & , j = 1 \wedge m > 1 \\ c(x_{j-1}, m) + p(x_j, m) & , j > 1 \wedge m = 1 \\ \max\{c(x_{j-1}, m), c(x_j, m-1)\} + p(x_j, m) & , j > 1 \wedge m > 1 \end{cases},$$

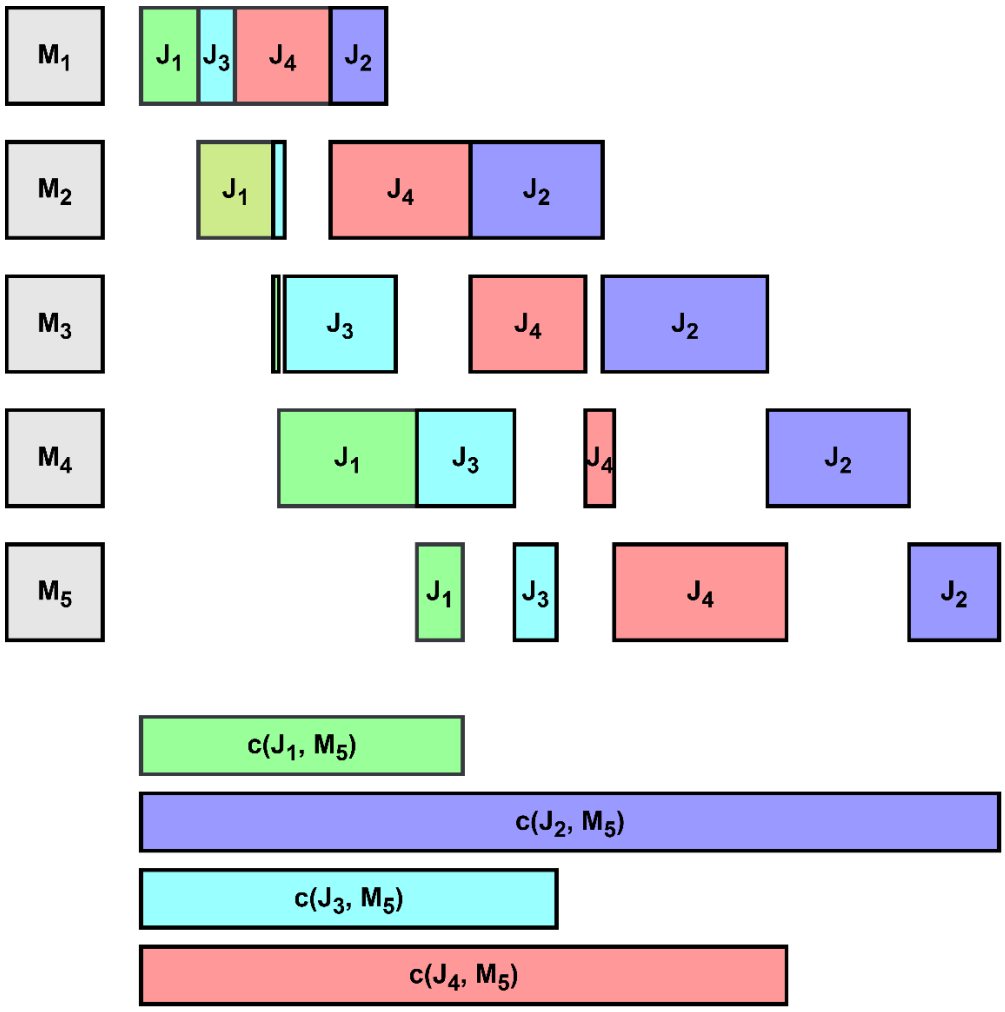
gdzie $p(x_j, m)$ jest czasem wykonania x_j-tego zadania na m-tej maszynie.

Pliki wejściowe do problemu PFSP zawierają m.in. liczbę zadań (J), liczbę maszyn (M) oraz macierz o wymiarach $M \times J$ zawierającą czasy wykonania j-tego zadania na m-tej maszynie.

Plik wejściowy do testów:

```
number of jobs, number of machines, initial seed, upper bound and lower bound :
        4        5  -      -      -
processing times :
 30 29 19 50
 39 69  6 73
  3 86 58 60
 72 74 51 15
 24 47 22 90
```

Powyższa, testowa instancja problemu PFSP składa się z 5 maszyn i 4 zadań. Na przykład, czas realizacji trzeciego zadania na drugiej maszynie wynosi 6. Optymalną sekwencją dla testowej instancji jest $x^* = (1, 3, 4, 2)$, dla której wartość funkcji celu wynosi $f(x_1, x_2, x_3, x_4) = c(1, 5) + c(3, 5) + c(4, 5) + c(2, 5) = 168 + 217 + 337 + 448 = 1170$. Optymalne rozwiązanie dla testowej instancji przedstawione jest na Rysunku 2.



Rysunek 2. Wizualizacja optymalnego rozwiązania dla testowej instancji

Docelowo interesują nas 6 różnych instancji zbioru Taillard (https://github.com/chneau/go-taillard/tree/master/pfsp/instances).

Instancje – łatwe (3):

tai20_5_0, tai20_10_0, tai20_20_0.

Instancje – średnie (2):

tai100_10_0, tai100_20_0.

Instancje – trudne (1):

tai500_20_0.

Osobnik dla problemu PFSP może być **reprezentowany** jako sekwencja zadań, np. 35124. Pierwszym zadaniem uruchamianym na każdej maszynie jest zadanie nr 3, potem realizujemy zadanie nr 5, następnie zadanie nr 1 itd. W takiej reprezentacji każde z zadań musi wystąpić tylko raz.

Operator **mutacji** dla powyższej reprezentacji wektorowej sprowadzać się może do zamiany losowo dwóch zadań (tzw *swap)*. Przykład działania mutacji przedstawiono na Rysunku 3. Polega ona na wyznaczeniu dwóch genów w chromosomie i zamianie ich wartości. Na przykładzie pokazano zamianę pozycji drugiej (zadanie 6) i przedostatniej (zadanie 3). W wyniku wymiany powstaje nowy osobnik.

Można także rozpatrzyć mutację typu inwersja – przykład jej działania przedstawiono na Rysunku 3. Jej działanie polega na wyznaczeniu losowo dwóch miejsc w osobniku i w ramach tego obszaru „odwracamy” kolejność elementów (zadań).

Swap – |5671234| → |5371264|
Inwersja |5671234| → |5321764|

Rysunek 3. Schemat działania mutacji typu SWAP i Inwersja

Operator **krzyżowania** łączy dwa osobniki tworząc potomny (lub dwa, zależnie od operatora). W najprostszej postaci krzyżowanie znajduje losowy punkt przecięcia i dla potomka pierwszą część bierze od jednego rodzica, drugą część od drugiego. Uwaga! Po tej operacji należy sprawdzić czy nie brakuje jakiegoś zadania i/lub które występuje podwójnie w osobniku potomnym. Wymagana jest procedura „naprawy” osobnika. Przykłady operatorów krzyżowania (OX, CX, PMX) zostały przedstawione na wykładzie.

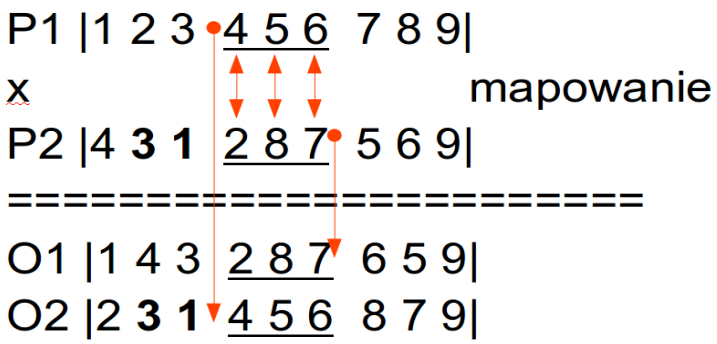
Operator krzyżowania OX (Ordered Crossover) polega na wyznaczeniu dwóch miejsc w jednym osobniku rodzicielskim. Ten podciąg będzie wpisywany bez zmian do osobnika potomnego bez zmian. Z drugiego osobnika rodzicielskiego będzie kopiowana kolejność zadań. Przykład działania operatora krzyżowania OX przedstawiono poniżej.

Podciąg od jednego rodzica, uzupełnij kolejnością od drugiego

P1 |123456789| x P2 |574913628|

→ O: |793456128|

Operator krzyżowania PMX (Partially Matched Crossover) jest nieco bardziej skomplikowany w działaniu. Wyznacza się dwa losowe punkty, które będą wyznaczać podsekcję krzyżowania. W osobników rodzicielskich wyznacza się sekcję, która jest mapowana, wyznaczane są zadania sobie odpowiadające. W osobnikach potomnych sekcja mapowania jest przepisywana od jednego osobnika rodzicielskiego, a brakujące zadania przepisywane są w kolejności z drugiego rodzica
Przykład działania operatora krzyżowania PMX przedstawiono poniżej:



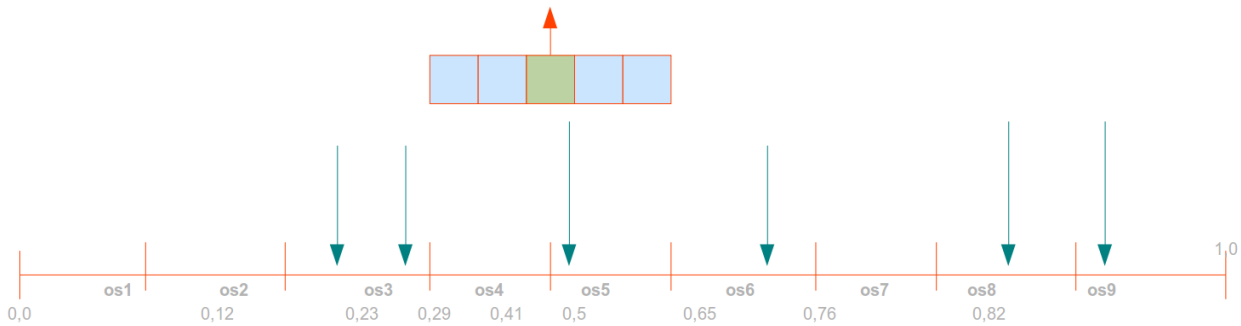
Ciekawym przykładem operatora krzyżowania jest CX (Cycle Crossover). Zasada działania tego operatora polega na zestawieniu obu rozwiązań rodzicielskich i wyznaczeniu „cyklu”, czyli które

zadania odpowiadają sobie pozycją. W dalszych krokach buduje się osobnik potomny, który oparty jest na zadaniach cyklu odpowiednio z jednego z rodziców. Pozostałe zadania uzupełnia się wg kolejności zadań drugiego rodzica. Przykład działania operatora krzyżowania CX przedstawiono poniżej:

```
P1 |2 7 4 3 1 5 6 9 8|
P2 |1 2 3 4 5 6 7 8 9|
----cykl 1-2-7-6-5-1-------->
O  | 2 7 _ _ 1 5 6 _ _ |  z P1
O  | _ _ 3 4 _ _ _ 8 9 |  z P2
=====================
```

Operatory krzyżowania opisane są literaturze podanej na końcu dokumentu. Najbardziej polecaną jest pozycja [4].

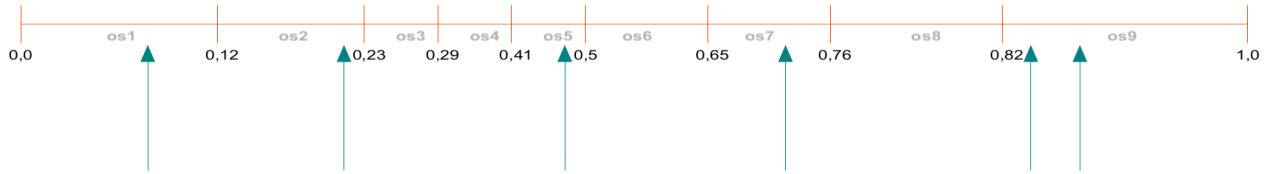
Operator **selekcji** stanowi bardzo ważny element EA – to on decyduje o ciśnieniu selekcyjnym. Jeśli ciśnienie jest zbyt duże populacja szybko się ujednolica i utyka w lokalnym optimum. Jeśli ciśnienie jest za małe (lub go brak), to EA ma trudność ze zbieżnością. Najczęściej stosowanymi metodami selekcji jest turniej i ruletka.
Metoda selekcji **turnieju** (patrz Rysunek 4) polega na losowym wyborze N[1] osobników z populacji i wskazanie w wyniku osobnika z najlepszym przystosowaniem. Jeśli w $N=1$ wybór osobników jest losowy i brak jakiegokolwiek ciśnienia selekcyjnego. Dla N równego rozmiarowi populacji metoda turniejowa bliższa jest metodzie elitarnej (opis poniżej), czyli wyboru najlepszego osobnika z populacji.



Rysunek 4. Schemat działania selekcji turniejowej (n=5)

[1] N jest parametrem metody Turniej -- może mieć wartość liczbową lub procentową w stosunku do rozmiaru populacji na której pracuje, np. 5%

Inna metoda selekcji, tzw. **ruletka** (patrz Rysunek 5), która pozwala na wybór osobnika z prawdopodobieństwem proporcjonalnym do jego jakości (funkcji przystosowania). Warto wskazać, że ruletka dla problemów kombinatorycznych przy dużych wartościach funkcji oceny i małych (%) różnicach w wartościach może powodować za małe ciśnienie selekcyjne – wtedy należy rozważyć skalowania funkcji przystosowania, gdzie uwypukla się różnice w wartościowaniu funkcji oceny dla wskazanej populacji.



Rysunek 5. Schemat działania selekcji ruletki

Dodatkowy parametrem jest **elitaryzm**, który czasem też jest nazywamy metodą selekcji. W praktyce jest to wybór *M* najlepszych osobników z populacji. Wartość *M* – podobnie jak wartość *N* dla turnieju może być arbitralna lub procentowa – określa ile osobników bez zmian przechodzi do następnego pokolenia. Parametr elitarny może być bardzo przydany dla dużych populacji (nie gubimy najlepszych) jednak dla małych jest bardzo zgubne, bo może „kotwiczyć" populację w lokalnych optimach.

Metoda **inicjalizacji** jest kluczowa dla działania EA nie tylko dla początku procesu przeszukiwania przestrzeni rozwiązań. Za mało różnorodne osobniki początkowe mogą szybko spowodować utknięcie EA w lokalnym optimum. Z tego względu jako metodę inicjalizacji zwykle używa się losowych punktów startowych. Można też zastosować metody, które wstępnie przeszukają przestrzeń i dadzą EA lepsze punkty startowe. Przykładem takiej metody może być algorytm zachłanny (opisany poniżej), który buduje rozwiązanie lepsze od losowego.

Realizacja ćwiczenia

Zadanie ma kilka celów, które mogą być sprecyzowane jako:

- Zapoznanie się z metaheurystyką – algorytmy ewolucyjne
- Określenie problemu optymalizacyjnego do rozwiązania: permutacyjny problem przepływowy szeregowania zadań
- Algorytm zachłanny -- implementacja
- Inicjacja – np. losowa i zachłanna
- Zbudowanie algorytmu genetycznego: osobnik, funkcja oceny, zarządzanie populacją
  - krzyżowanie – odpowiednie dla rodzaju problemu
  - mutacja – odpowiednie dla rodzaju problemu,
- Zbudowanie innej metaheurystyki Tabu Search lub Symulowane Wyżarzanie
- Implementacja metod w dowolnym języku obiektowym. Sugeruje się używanie języków programowania: C/C++, Java, Python lub C#.
- Zbadanie wpływu wartości dla różnych parametrów (prawd. mutacji Pm, krzyżowania Px, selekcji, rozmiar populacji pop_size, liczba pokoleń gen) na efektywność i skuteczność algorytmu ewolucyjnego
- Sporządzenie sprawozdania z ćwiczenia
- Pokazanie na wykresach zmianę wartości przystosowania (wartość optymalizowanej funkcji celu) w poszczególnych pokoleniach: najlepszy osobnik, średnia wartość w populacji i najgorszy osobnik

- Porównaj działanie algorytmu ewolucyjnego z wybranymi przez siebie, nieewolucyjnymi metodami optymalizacji (sugerowana metoda losowego przeszukiwania i algorytm zachłanny).
- Proszę zwrócić uwagę na uzyskany wynik, czas działania oraz liczbę wartościowań optymalizowanej funkcji celu. Proszę pokazać i omówić najciekawsze wyniki.

Raport z ćwiczenia powinien zawierać wszystkie punkty wymagane w realizacji zadania.

Implementacja – wskazówki realizatorskie

Przy realizacji ćwiczenia sugeruje się utworzyć program komputerowy, który swoimi funkcjonalnościami obejmie:

- wczytywanie pliku z danymi, zapis problemu do pamięci komputera (logger, problem),
- oceny rozwiązania (osobnika) w kontekście wybranego problemu,
- zarządzanie parametrami algorytmu (konfiguracja),
- sterowania przebiegiem metaheurystyk (EA),
- przechowywania rozwiązania (osobnik) i operatorów genetycznych (krzyżowanie, mutacja),
- zarządzania populacją – w szczególności selekcja, inicjalizacja oraz tworzenie nowej,
- logowania postępów metody (logger) do pliku zewnętrznego (format csv?)

Kod programu powinien być na zaprojektowany na tyle elastycznie aby możliwe było użycie go przy realizacji projektu głównego

Do realizacji zadania potrzebne jest **porównanie** wyników EA do innych nieewolucyjnych metod. Wskazane są dwie metody: losowa i zachłanna. W przypadku losowej, wystarczy uruchomić metodę tyle razy ile jest rozwiązań przejrzanych przez EA (liczba pokoleń x liczba osobników) aby je porównać. W przypadku algorytmu zachłannego, jest on deterministyczny (zawsze daje to samo rozwiązanie) i opiera się na sekwencyjnej konstrukcji rozwiązania opierając się na lokalnej informacji – zawsze zadanie, z najmniejszą wartością $c(x_j, M)$. Tutaj, podobnie jak w algorytmie losowym, należy powtórzyć działanie dla różnych punków startowych.
Wyniki dla działania metod należy uśrednić.

Z punktu widzenia programowania sugeruje się rozważyć czy przy tworzeniu osobników czy potrzebujemy **kopii głębokiej** czy też wystarczy kopia płytka? Dla optymalności kodu proszę też rozważyć jak kodujemy osobnika (czy/jaka kolekcja jest użyta) – będzie to miało bardzo duży wpływ na szybkość działania kodu. Podobne rozważania proszę zrobić przy implementacji turnieju – proszę rozważyć która (i czy?) kolekcja w implementacji może być przydatna?

Raportowanie wyników i sugestie związane z ich analizą

Algorytm Ewolucyjny ma element niedeterminizmu i zwracane wyniki za kolejnymi uruchomieniami mogą się różnić. Z tego powodu wyniki badań przeprowadzonych przy użyciu wskazanych plików muszą być powtórzone 10-krotnie, uśrednione i podane odchylenie standardowe.

Dla każdego wyniku/wykresu należy podać użytą konfigurację metody: rozmiar populacji (*pop_size*), liczba pokoleń (*gen[erations]*), prawdopodobieństwo krzyżowania (*Px*), oraz prawdopodobieństwo mutacji (*Pm*). Dodatkowo, jaki typ krzyżowania/mutacji/selekcji został użyty.

W przypadku, gdy wymaga ona dodatkowych parametrów, np. rozmiar turnieju (*Tour*), tę wartość także należy podać w konfiguracji.

Dla przedstawienia wyników badań sugeruje się użycia tabelki zbiorczej formatu jak Tabela 1.

Tabela 1. Format sugerowany do przedstawienia wyników działania metaheurystyk na przykładzie EA

instancja	Alg.losowy [10k]				Alg.zachłanny [J]				Alg.Ewolucyjny [10x]				TS lub SA [10x]		
	best	worst	avg	std	best	worst	avg	std	best	worst	avg	std	Best	worst	avg
xyz2															
...	...	...	...	...	...	...	...	...	...	...	...	...	...		

Gdzie:

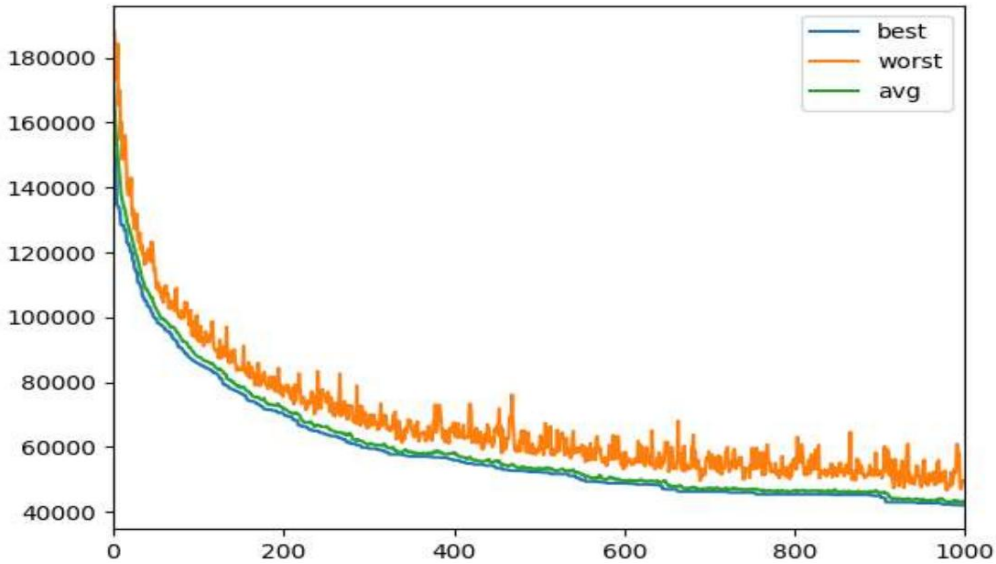
Alg.losowy [10k] oznacza 10.000 uruchomień algorytmu losowego
Alg.zachłanny [*J*] oznacza *J* uruchomień algorytmu zachłannego[2]
Alg.Ewolucyjny [10x] oznacza 10-krotne uruchomienie algorytmu ewolucyjnego i podanie wartości statystycznych dla tych uruchomień.
TS lub SA [10x] oznacza wyniki innej metaheurystyki (Tabu Search lub Symulowane Wyżarzanie); 10-krotne uruchomienie metody i podanie wartości statystycznych dla tych uruchomień.

*best** – oznacza najlepszą znalezioną wartość
worst – oznacza najgorszą znalezioną wartość
avg – oznacza średnią znalezioną wartość
std – oznacza wartość odchylenia standardowego liczonego przy średniej (*avg*)

Uwaga: wartość *best** dla algorytmu ewolucyjnego jest pobierana jako najlepsza wartość z 10 uruchomień. Średnia (avg) oznacza wartość liczoną z najlepszego rozwiązania z każdego uruchomienia. Na tej podstawie można utworzyć wykres jak na Rysunku 6.

Tabela 1. przestawia skuteczność zbadanych rozwiązań, czyli jak dobre rozwiązania (średnio) jesteśmy w stanie uzyskać uruchamiając metodę. Wykres z Rysunku 6 pozwala na wyciągnięcie bardzo wielu cennych wskazówek co do działania i konfiguracji algorytmu ewolucyjnego.

[2] W przypadku problemu PFSP należy zauważyć, że mamy „tylko” *J* zadań. Algorytm zachłanny jest deterministyczny i wystarczy aby każde zadanie było pierwszym wykonywanym zadaniem na każdej maszynie i ograniczamy się do *J* uruchomień algorytmu zachłannego.



Rysunek 6. Przykład działania EA w rozwiązywaniu wybranego problemu

Na Rysunku 6 przedstawiono wykres poprawnego działania skonfigurowanego algorytmu ewolucyjnego w rozwiązywaniu problemu TSP (problem komiwojażera, ang. *Traveling Salesman Problem*). Najlepszy osobnik (*best*) schodzi bardzo dynamicznie, choć w wykresie widać „tąpnięcia", które wskazują że występują sytuacje gdy najlepszy osobnik w populacji jest gubiony (nie przechodzi do następnej). W przypadku najgorszego osobnika w populacji (*worst*) wykres jest bardziej dynamiczny, co wskazuje na pozytywne zachowanie i ciągłe przeszukiwanie przestrzeni. Odległość wykresu najgorszego i najlepszego wskazuje, że w populacji występuje wystarczająca różnorodność. Trzeci wykres średniego przystosowania (*avg*) wskazuje, że różnorodność jest na wystarczającym poziomie.

Ocena realizacji ćwiczenia

Realizacja zadania jest oceniana na podstawie składowych wg poniższego schematu punktów:

1pkt	Zbadanie działania EA na 7 plikach – instancje podane wraz z treścią. Testy muszą być przeprowadzone na: 3 łatwe, 2 średnie i 1 trudna instancje + jedna instancja *extra* (do wstępnego przebadania).
1pkt	Implementacja i zbadanie algorytmu losowego
1pkt	Implementacja i zbadanie algorytmu zachłannego
2pkt	Implementacja i zbadanie innej metaheurystyki
3pkt	Zbadanie wpływu konfiguracji obu metaheurystyk na ich skuteczność/efektywność
2pkt	Porównanie skuteczności wszystkich metod

Uwaga: Przy badaniu wyników dla danej metody należy brać pod uwagę przynajmniej 10 uruchomień i uśrednianie wyniku (podajemy wartość średniej i odchylenie standardowe).

Podpowiedzi

Wstępne parametry przydane do strojenia EA:

pop_size=100, gen=100, Px=0,7, Pm=0,1, Tour=5

Uwaga! optymalne wartości parametrów będą się różnić, w zależności od użytych operatorów oraz rozwiązywanego przypadku wybranego problemu.

Dla wartości prawdopodobieństwa mutacji należy rozważyć użycie prawdopodobieństwa mutacji (*Pm*) w zależności od:

- **osobnika** dla mutacji działających na osobniku. Przykładem takiej mutacji jest operator inwersji, gdzie niejawnie zakłada się jej tylko jednorazowe działanie na/w danym osobniku. Wtedy *Pm=0,1* oznacza, że statystycznie 10% osobników w populacji będzie mutowane. Dla przykładu, w populacji 100 osobników około 10 osobników będzie zmutowane.
- **genu** dla mutacji działających tylko na małym fragmencie osobnika. Przykładem takiej mutacji może być operator *swap* (w PFSP zamiana kolejności dwóch zadań). W takim wypadku sugeruje się rozpoczęcie badania od Pm=0,01. Przykładowo, dla problemu PFSP z 100 zadań i 100 osobników w populacji, średnio będzie zmienione około 200 genów (100*100*0,01 operacji na 2 genach).

Mając logowanie do pliku **.csv* wartości statystyczne dla każdego pokolenia w formacie:

<nr_pokolenia; najlepsza_ocena; średnia_ocen; najgorsza_ocena;;;>

można „za darmo” uzyskać wykresy w Excelu (jak Rysunek 6). Wtedy można skupić się na merytoryce zadania, mniej na interfejsie, GUI, interaktywnych wykresach itp.

Realizacja zadania

Proponuje się systematyczną realizację zadania wg następującej kolejności etapów.

<u>Zajęcia 1.</u>

Zapoznanie się z problemem.
Implementacja loadera.
Implementacja losowego osobnika PFSP.
Implementacja funkcji f(x)
Implementacja algorytmu zachłannego

<u>Zajęcia 2.</u>

Implementacja operatorów mutacji i krzyżowania.
Implementacja selekcji i zarządzania populacją w algorytmie ewolucyjnym.
Implementacja narzędzi logowania → wykresy.
Implementacja innej metaheurystyki (SA lub TS)
Wstępne strojenie metaheurystyk.
Badanie wpływu wartości parametrów na skuteczność/efektywność algorytmu genetycznego.
Przygotowanie sprawozdania: tabelki, wykresy, wnioski.

Pytania pomocnicze

1. Jak selekcja (rozmiar turnieju *Tour*) wpływa na działanie EA? Co się dzieje jeśli mamy *Tour=1* a co się dzieje jak *Tour=pop_size*?
2. Czym się różni działanie EA z Tour=1 od EA z Tour>1?
3. Czy ruletka działa poprawnie dla problemu PFSP?
4. Czy mutacji może być za mało/dużo?
5. Jaką rolę pełni operator mutacji?
6. Czy krzyżowania może być za mało/dużo?
7. Jaką rolę pełni operator krzyżowania?
8. Co się dzieje jeśli wyłączymy krzyżowanie i/lub mutację?
9. Czy selekcja elitarna jest potrzebna?
10. Jak rozmiar populacji i liczba osobników wpływa na efektywność/skuteczność EA?

Uwaga: przy badaniu efektywności metody proszę brać pod uwagę „liczbę urodzeń", tj. liczbę osobników odwiedzonych przez EA. W praktyce można to sprowadzić się to do określenia wartości *pop_size*gen*

Literatura

[1] Arabas J. Wykłady z algorytmów ewolucyjnych (http://staff.elka.pw.edu.pl/~jarabas/ksiazka.html)

[2] Goldberg D. Algorytmy genetyczne i ich zastosowanie, WNT

[3] Michalewicz Z. Algorytmy genetyczne + struktury danych = programy ewolucyjne, WNT.

[4] Krunoslav Puljić, Robert Manger, „Comparison of eight evolutionary crossover operators for the vehicle routing problem", MATHEMATICAL COMMUNICATIONS Math. Commun. 18(2013), 359–375

[5] Potvin, Jean-Yves. 1996. Genetic algorithms for the the traveling salesman problem, Annals of Operations Research, Volume 63, pages 339-370.

[6] E. Taillard, 1993. Benchmarks for basic scheduling problems, European Journal of Operational Research, Volume 64, Issue 2, Pages 278-285.

[7] Peter A.N. Bosman, Ngoc Hoang Luong, and Dirk Thierens. 2016. Expanding from Discrete Cartesian to Permutation Gene-pool Optimal Mixing Evolutionary Algorithms. In Proceedings of the Genetic and Evolutionary Computation Conference 2016 (GECCO '16). Association for Computing Machinery, New York, NY, USA, 637–644.

[8] PFSP instances: https://github.com/chneau/go-taillard/tree/master/pfsp/instances